LegacyLens -- RAG System for Legacy Codebases

Technology Stack

- **Backend**: Python 3.11+ / FastAPI
- **Vector DB**: Pinecone (managed free tier -- zero ops overhead, easy deployment)
- **Embeddings**: OpenAI `text-embedding-3-small` (1536 dims, best cost/quality ratio)
- **LLM**: GPT-4o-mini (fast + cheap for most queries), GPT-4o (complex explanations)
- **RAG Framework**: LangChain (flexible pipelines, native Pinecone + OpenAI integrations)
- **Frontend**: React + Vite + Tailwind CSS + `react-syntax-highlighter`
- **Target Codebase**: LAPACK (Fortran linear algebra library)
- **Deployment**: Railway (backend) + Vercel (frontend)

Project Structure

```
legacylens/
├── backend/
│   ├── app/
│   │   ├── main.py              # FastAPI app + routes
│   │   ├── config.py            # Settings via pydantic-settings
│   │   ├── ingestion/
│   │   │   ├── scanner.py       # Recursive file discovery
│   │   │   ├── chunker.py       # Fortran syntax-aware chunking
│   │   │   ├── embedder.py      # Batch embedding generation
│   │   │   └── pipeline.py      # Ingestion orchestrator
│   │   ├── retrieval/
│   │   │   ├── search.py        # Pinecone similarity search
│   │   │   ├── reranker.py      # Result re-ranking
│   │   │   └── context.py       # Context assembly + expansion
│   │   ├── generation/
│   │   │   ├── llm.py           # LLM answer generation
│   │   │   └── prompts.py       # Prompt templates
│   │   └── features/
│   │       ├── explain.py       # Code explanation
│   │       ├── docgen.py        # Documentation generation
│   │       ├── patterns.py      # Pattern detection
│   │       ├── dependencies.py  # Dependency mapping
│   │       └── business.py      # Business logic extraction
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── QueryInput.tsx
│   │   │   ├── ResultsPanel.tsx
│   │   │   ├── CodeSnippet.tsx
│   │   │   └── FeaturePanel.tsx
│   │   ├── App.tsx
│   │   └── main.tsx
│   └── package.json
```

```
├── scripts/
│   └── ingest.py                # CLI script to run ingestion
├── docs/
│   ├── architecture.md
│   ├── cost-analysis.md
│   └── pre-search.md
└── README.md
```

Phase 1: MVP (Target: 24 Hours)

The goal is a working end-to-end RAG pipeline: ingest -> embed -> store -> search -> answer.

1.1 Project Scaffolding

- Initialize Python project with `requirements.txt` (fastapi, uvicorn, langchain, langchain-openai, langchain-pinecone, pinecone-client, python-dotenv, pydantic-settings)
- Initialize React + Vite frontend (`npm create vite@latest frontend -- --template react-ts`)
- Set up `.env` for API keys (OPENAI_API_KEY, PINECONE_API_KEY, PINECONE_INDEX)
- Download LAPACK source from GitHub: `git clone https://github.com/Reference-LAPACK/lapack.git data/lapack`

1.2 File Discovery + Preprocessing (`scanner.py`)

- Recursively walk `data/lapack/SRC/` , `data/lapack/BLAS/SRC/` for `.f` , `.f90` files
- Normalize encoding (handle any non-UTF8)
- Extract file-level metadata: path, LOC, language variant (F77 vs F90)
- Target: index all `.f` and `.f90` files from LAPACK SRC + BLAS SRC directories

1.3 Fortran-Aware Chunking (`chunker.py`)

- **Primary strategy**: Split on `SUBROUTINE` and `FUNCTION` boundaries using regex:

```
# Fortran function/subroutine pattern
pattern = r'(?:^|\n)\s*(SUBROUTINE|FUNCTION|PROGRAM|BLOCK DATA)\s+\w+'
```

- Each chunk = one complete subroutine/function including its doc comment header
- **Fallback**: Fixed-size chunking (800 tokens) with 200-token overlap for non-routine code
- **Metadata per chunk**: file_path, start_line, end_line, routine_name, routine_type, file_name
- Preserve the doc comment block that precedes each routine (LAPACK has extensive header comments)

1.4 Embedding Generation (`embedder.py`)

- Use OpenAI `text-embedding-3-small` via LangChain's `OpenAIEmbeddings`
- Batch embed chunks (max 2048 per batch per OpenAI limits)
- Prepend metadata to chunk text before embedding: `"File: {path} | Routine: {name} | Type: {type}\n{code}"`
- Rate limit handling with exponential backoff

1.5 Vector Storage in Pinecone (`pipeline.py`)

- Create Pinecone index (1536 dims, cosine similarity)

- Upsert all chunks with metadata fields for filtering
- Verify: query with a sample and confirm retrieval works

1.6 Retrieval Pipeline (`search.py` , `context.py`)

- Convert user query to embedding using the same model
- Pinecone similarity search with top-k=10
- Return chunks with metadata (file path, line numbers, routine name, similarity score)
- Context assembly: include surrounding lines from original file for expanded context

1.7 Answer Generation (`llm.py` , `prompts.py`)

- LangChain with GPT-4o-mini
- Prompt template:

```
  You are a Fortran/LAPACK code expert. Answer the question using ONLY the code
context provided.
  Always cite file paths and line numbers. If the context doesn't contain the answer,
say so.

  Context: {retrieved_chunks}
  Question: {query}
```

- Return structured response: answer text + source references

1.8 API Routes (`main.py`)

- `POST /api/query` -- main query endpoint (question -> answer + sources)
- `POST /api/search` -- raw semantic search (question -> ranked chunks)
- `GET /api/stats` -- index stats (files indexed, total chunks, etc.)
- `POST /api/ingest` -- trigger ingestion (admin endpoint)
- CORS middleware for frontend

1.9 Minimal Frontend

- Single-page app: query input box + results panel
- Display: answer text, source code snippets with syntax highlighting, file paths, relevance scores
- Use `react-syntax-highlighter` for Fortran code display
- Tailwind for styling (clean, minimal UI)

1.10 MVP Deployment

- Backend: Dockerize FastAPI app, deploy to Railway
- Frontend: Deploy to Vercel
- Verify end-to-end: query from deployed frontend -> deployed backend -> Pinecone -> response

---

Phase 2: Refinement + Features (Days 2-3)

2.1 Improved Chunking

- Hierarchical chunking: file-level summaries + routine-level chunks
- Better comment extraction: parse LAPACK's structured header comments (PURPOSE, ARGUMENTS, etc.)
- Handle COMMON blocks and INCLUDE statements

2.2 Query Processing Enhancements

- Query expansion: use LLM to rephrase query into Fortran-specific terms (e.g., "matrix multiply" -> "DGEMM SUBROUTINE matrix multiplication")
- Intent detection: is the user asking for explanation, search, or dependency info?

2.3 Re-ranking (`reranker.py`)

- After initial Pinecone retrieval (top-20), re-rank using cross-encoder or LLM-based scoring
- Score based on: relevance to query + code completeness + metadata match

2.4 Code Understanding Features (implement 5)

Feature 1: Code Explanation (`explain.py`)

- Send selected routine to GPT-4o with structured prompt
- Return: purpose, inputs/outputs, algorithm description, complexity

Feature 2: Documentation Generation (`docgen.py`)

- Generate missing or enhanced documentation for routines
- Output formatted docstrings with parameter descriptions

Feature 3: Pattern Detection (`patterns.py`)

- Given a code snippet, find similar patterns using embedding similarity
- "Show me other routines that do error checking like this one"

Feature 4: Dependency Mapping (`dependencies.py`)

- Parse CALL statements and EXTERNAL declarations
- Build call graph: "What does DGESV call? What calls DGESV?"
- Store dependency edges as metadata in Pinecone

Feature 5: Business Logic Extraction (`business.py`)

- Identify and explain the mathematical/business rules encoded in routines
- "What algorithm does this routine implement?"

2.5 Frontend Enhancements

- Tabbed interface: Search | Explain | Dependencies | Patterns
- Code snippet expansion (click to see full file context)
- Syntax-highlighted Fortran with line numbers
- Confidence/relevance score badges on results
- Loading states and error handling

Phase 3: Polish + Deployment (Days 4-5)

3.1 Performance Optimization

- Cache frequently queried embeddings
- Precompute file-level summaries for faster broad queries
- Optimize chunk sizes based on retrieval quality testing
- Target: <3s end-to-end query latency

3.2 Evaluation + Metrics

- Create test query set (10-15 queries from the PDF's testing scenarios)
- Measure retrieval precision@5 (target >70%)
- Log query latency, token usage, retrieval scores
- Document failure modes (what queries don't work well)

3.3 Documentation Deliverables

- **RAG Architecture Doc** (1-2 pages): vector DB selection rationale, embedding strategy, chunking approach, retrieval pipeline flow, failure modes, performance results
- **Cost Analysis**: actual dev spend + projections for 100/1K/10K/100K users
- **Pre-Search Document**: completed checklist from the PDF appendix
- **README**: setup guide, architecture overview, deployed link

3.4 Final Deployment

- Environment variables properly configured
- Health checks and error handling
- HTTPS on deployed URLs
- Demo-ready state

3.5 Demo Video (3-5 min)

- Show ingestion process
- Demo various query types from the testing scenarios
- Show code understanding features
- Highlight architecture decisions

Key Implementation Details

Fortran Chunking Regex (critical for MVP)

```
import re

ROUTINE_PATTERN = re.compile(
    r'(?:
(?:RECURSIVE|PURE|ELEMENTAL|INTEGER|REAL|DOUBLE\s+PRECISION|COMPLEX|CHARACTER|LOGICAL)
    r'(?:SUBROUTINE|FUNCTION|PROGRAM|BLOCK\s+DATA)\s+(\w+)',
    re.IGNORECASE | re.MULTILINE
)
```

Pinecone Index Setup

```
from pinecone import Pinecone, ServerlessSpec

pc = Pinecone(api_key=PINECONE_API_KEY)
pc.create_index(
    name="legacylens",
    dimension=1536,
    metric="cosine",
    spec=ServerlessSpec(cloud="aws", region="us-east-1")
)
```

LangChain RAG Chain (core retrieval + generation)

```
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_pinecone import PineconeVectorStore
from langchain.chains import RetrievalQA

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = PineconeVectorStore(index_name="legacylens", embedding=embeddings)
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vectorstore.as_retriever(search_kwargs={"k": 10}),
    return_source_documents=True,
)
```

Critical Path for MVP

1. Get LAPACK cloned and files discovered
2. Chunking working (even simple fixed-size is fine for day 1)
3. Embeddings generated and stored in Pinecone
4. Basic query returning relevant chunks
5. LLM generating answer from chunks
6. Minimal web UI showing results
7. Deployed to Railway + Vercel